1 Stochastic Model and Quality Control

This document covers the *measurement* noise of the estimator. It describes the strategies that fill the observation-noise covariance $\boldsymbol{R}$, and the a-posteriori quality-control (QC) layer that tests an epoch's residuals against that $\boldsymbol{R}$. The $\boldsymbol{R}$ here has the block-diagonal structure of the observation-equations noise model, and this document gives the variance each strategy puts in it. The QC layer uses the post-fit residual statistics that the filter already computes for outlier rejection, and presents them in the classical geodetic detection theory of Baarda and Teunissen [7]. The network estimator has no Kalman residuals of this kind; its own quality control (a robust variance factor, the RAIM tiers, the block and carry statistics and the measure-only instruments) is treated in the later subsections, from Section 1.2 onward.

Two topics are outside this document and are only cross-referenced. The first is the reaction to a detected blunder: the standardised-residual outlier path ($|s| > 6$), the Huber re-weighting, and the EWMA inflation of $\boldsymbol{R}$. These are in the Kalman-filter gating section. This document explains how the QC statistics drive an *alternative* gate but does not repeat the dispatch. The second is the process-noise covariance $\boldsymbol{Q}$ (clock random walk, ZWD, ambiguity, IMU bias), which belongs to the dynamics and is described with the smoother (the process-noise section). Everything below is measurement noise.

1.1 The stochastic-model dispatch

Every undifferenced observation (r, i, s) is assigned a scalar variance by a strategy object selected from the `StochasticModelType` enum (`include/config/GnssConfigTypes.hpp`, the four-way {`Constant`, `Elevation`, `SNR`, `SatelliteBlock`} set). The dispatch is in `src/core/model/StochasticModel.cpp`. Each strategy returns a variance in m^2, which becomes one diagonal entry of $\boldsymbol{R}$. The three strategies that use geometry share a low-elevation guard. When $\sin\theta \le 10^{-6}$ the elevation amplification is skipped and the unamplified variance is returned, so a satellite at the horizon does not receive a numerically infinite variance (`computeElevation`). The `Constant` strategy has no elevation term. It returns its a-priori variance directly (`computeConstant`) and never reaches the guard.

(a) Constant. This strategy returns the a-priori variance of unit weight unchanged,

$$\sigma^2_{r,i,s} = \sigma^2_{0,\mathrm{type}(s)}, \tag{1}$$

with σ_0 drawn from `filter.measurement_noise` (`computeConstant`). The other strategies scale this value.

(b) Elevation. The classical elevation-dependent model scales the zenith-referenced variance by an inverse power of $\sin\theta$,

$$\boxed{\sigma^2_{r,i,s} = \sigma^2_{\mathrm{zen}} \left(\sin\theta^i_r\right)^{-2p_{\mathrm{type}}}, \qquad p_P = p_\Phi = 1.0.} \tag{2}$$

The shipped defaults of `filter.code_elevation_power` and `filter.phase_elevation_power` are both one, so the shipped model is *symmetric*: code and phase both follow the legacy $\csc^2\theta$ variance law (`computeElevation`). The exponent is $-2p_{\mathrm{type}}$ because the strategy works in variance, not σ:

$\left(\sin\theta\right)^{-2p} = \left(\sin\theta^{-p}\right)^2$. Rows whose observable code does not begin with `L` take the code power. That is every row except carrier phase, including Doppler and the fallback paths with an empty `obs_type`.

Both exponents are configurable per observable class. The asymmetric setting $p_\Phi = 0.5$ is the one alternative with a physical argument, so the reasons it is an *option* and not the default are recorded here. It gives a $\csc\theta$ variance ($\csc^{1/2}\theta$ in σ), which amplifies much less at low elevation.

The argument for it has two parts. Phase noise near the horizon is dominated by carrier-loop SNR and by antenna phase-centre miscalibration, and neither follows the $\csc^2\theta$ law that governs code multipath and pseudorange path-length error. The milder amplification also keeps low-elevation phase within the useful range of the standardised-residual cycle-slip test, where a steeper law would deweight it until the test has no effect.

The argument against it is comparability: the code branch and every archived benchmark run at one. The loader therefore ships one (`GnssConfig::getPhaseElevationPower`, whose documentation gives the amplification ratios at a 10° mask).

(c) SNR (SIGMA-ε). When carrier-to-noise density is available the strategy uses the Hadás–Bosy variance-additive form [4]. Writing the measured (or synthesised) density as $\mathrm{C/N_0}$ in dB Hz,

$$\boxed{\sigma^2_{\mathrm{SNR}} = a^2 + b^2\, 10^{-\mathrm{C/N_0}/10},} \tag{3}$$

where a is the high-SNR floor ($\sigma \to a$ as $\mathrm{C/N_0} \to \infty$) and b scales the SNR-dependent contribution; both have separate code/phase values (`snr_params_.code_a/b`, `phase_a/b`). Adding *variances*, as in Equation (3), is the correct way to combine two independent noise sources. The legacy alternative adds standard deviations, $\sigma = a + b\,10^{-\mathrm{C/N_0}/10}$. `computeSnr` rejects it, because squaring it produces a cross-term $2ab\,10^{-\mathrm{C/N_0}/10}$ that corresponds to no noise process: it treats the two independent contributions as correlated.

When no `S...` observable is present the model keeps the same functional form. It *synthesises* a density from elevation, so the noise curve stays continuous (`synthesizeCn0FromElevation`):

$$\mathrm{C/N_0}(\theta) = 50 - 15\left(1 - \max(0, \sin\theta)\right) \ \ [\mathrm{dB\,Hz}], \tag{4}$$

which is a maximum of 50 dB Hz at zenith, falling by 15 dB at the horizon. The density is then clamped to the configured working range [`snr_floor_dbhz`, `snr_ceil_dbhz`]. Two refinements follow.

On code rows, a per-band scale s^2_{band} from the SatelliteBlock code table multiplies σ^2_{SNR}. Wider-bandwidth signals such as L5/E5a have $\sim\sqrt{10}$ less correlator noise in σ, a factor of $\sim 1/10$ in variance. Phase has no band scale, because PLL noise depends on the carrier-loop bandwidth, not the chip rate.

The Wieser hybrid step then passes the SNR variance through Equation (2). Pure SNR weighting under-weights multipath that is correlated with elevation, because the carrier loop can track cleanly while the range is corrupted. The elevation factor accounts for it.

(d) SatelliteBlock. The most detailed strategy keys the zenith σ on the transmitting satellite's ANTEX block type, then applies observable-specific frequency-band and tracking-attribute scales (`computeSatBlock`):

$$\boxed{\sigma^2_{r,i,s} = \left(\sigma_{\mathrm{block}(i),\,\mathrm{type}}\; s_{\mathrm{band}}\; s_{\mathrm{attr}}\right)^2 \left(\sin\theta\right)^{-2p_{\mathrm{type}}}.} \tag{5}$$

Here $\sigma_{\mathrm{block}(i),\mathrm{type}}$ is looked up by substring-matching the satellite's ATX type string against `block_code_std_m` / `block_phase_std_m` (falling back to `default_code_std_m` / `default_phase_std_m`

when no block matches); s_{band} is the per-band factor keyed on the RINEX band character (`obs_type[1]`); and s_{attr} is the tracking-attribute factor keyed on `obs_type[2]` via a *per-system* table, because a RINEX-3 attribute means different things in different constellations: a `C` attribute is the noisy GPS C/A code but the high-precision Galileo E1C pilot. The attribute lookup order is `[system][attr]` → `[default][attr]` → 1.0. Code and phase have separate band and attribute tables (`SatBlockNoiseParams::ObservableScale`). Chip-rate calibration governs code, while carrier-tracking calibration governs phase and varies much less between bands. As with the other geometry strategies, the block variance is finally amplified by Equation (2). Phase reaches this path only when `apply_to_phase` is set. Otherwise it takes the flat `default_phase_std_m` route in the top-level dispatch.

All four strategies fill diagonal entries only, so the undifferenced $\boldsymbol{R}$ is block-diagonal, as in the observation noise model. Off-diagonal correlation appears only after differencing (Section 1.5).

1.2 A-posteriori variance factor

The Kalman-side QC layer of this section and the next three is a diagnostic that is *off by default*. `KalmanFilter::update` calls `computeResidualQc_` only when `estimation.enable_variance_factor` or `estimation.enable_data_snooping` is set. The correlation-aware kernel of Section 1.5 also requires `estimation.enable_full_r_kernel`. All three default to false. With them off, the only outlier gate is the fixed $|s| > 6$ path of the gating section, the variance factor stays at its initial value of one, and the redundancy numbers, w-tests and minimal detectable biases below are not computed.

After an epoch's update, the post-fit residuals $\hat{\boldsymbol{v}}$ show whether the assumed $\boldsymbol{R}$ was correctly scaled. The a-posteriori *variance factor* (estimated variance of unit weight) is

$$\boxed{\hat{\sigma}_0^2 = \frac{\Omega}{\text{dof}}, \qquad \Omega = \hat{\boldsymbol{v}}^{\mathrm{T}} \boldsymbol{R}^{-1} \hat{\boldsymbol{v}} = \sum_i \frac{\hat{v}_i^{\,2}}{R_{ii}} \,\Bigg|_{\boldsymbol{R}\ \text{diag}}, \qquad \text{dof} = \operatorname{tr} \boldsymbol{\Theta},} \tag{6}$$

where Ω is the weighted sum of squared residuals and the degrees of freedom dof is the trace of the redundancy matrix $\boldsymbol{\Theta}$ (Equation (11) below). This is the *effective* redundancy of a recursively filtered epoch, not the $m-n$ of a single batch (`computeResidualQc_`, `src/core/filter/KalmanResidualQc.cpp`). $\boldsymbol{R}$ is in absolute units, so the a-priori variance of unit weight is $\sigma_0^2 = 1$. A well-scaled model gives $\hat{\sigma}_0^2 \approx 1$. $\hat{\sigma}_0^2 \gg 1$ means the assumed noise is too small (optimistic), and $\hat{\sigma}_0^2 \ll 1$ means it is too large (pessimistic).

The variance factor is used for *reporting only*. It never feeds back into the propagated covariance $\boldsymbol{P}$. When `enableVarianceFactor()` is set, the filter *logs* a copy of the position σ scaled by $\sqrt{\hat{\sigma}_0^2}$ and leaves the filter state unchanged (`reportStatisticalQc_` in `src/core/filter/KalmanPostFitQc.cpp`).

The same quantity has a batch form in the normal-equation accumulator of the PPP NEQ-export path (constructed only under `enableNeqExport()` in `Application::buildNeqAccumulator_`). Given normal matrix $\boldsymbol{N}$, right-hand side $\boldsymbol{u}$, and the pre-reduced $\boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l}$ term, the accumulator forms (`NeqAccumulator::solve`)

$$\Omega = \boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l} - \boldsymbol{u}^{\mathrm{T}}\hat{\boldsymbol{x}}, \qquad \text{dof} = n_{\text{obs}} - (n_{\text{params}} + n_{\text{local}}), \tag{7}$$

$$\hat{\sigma}_0^2 = \frac{\Omega}{\text{dof}}, \qquad \sigma_i = \sqrt{\hat{\sigma}_0^2 \left(\boldsymbol{N}^{-1}\right)_{ii}}. \tag{8}$$

Here $\hat{\boldsymbol{x}} = \boldsymbol{N}^{-1}\boldsymbol{u}$ is the least-squares solution. The redundancy subtracts the Schur-reduced local parameters n_{local} as well as the retained parameters n_{params}. The formal σ of each retained parameter

is the square root of the corresponding diagonal of the inverse normal matrix, scaled by $\hat{\sigma}_0^2$. This is the batch counterpart of the recursive dof in Equation (6).

The network estimator uses neither of these as its *operational* variance factor. The undifferenced network estimator does not form Ω/dof. It estimates a robust unit-weight σ as the scaled median of the standardised absolute residuals of its carried-ambiguity pool, $\tilde{v}_i = |\hat{v}_i|\sqrt{w_i}$,

$$\hat{\sigma}_{0,\mathrm{net}} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \mathrm{median}_i\,\tilde{v}_i\big), \tag{9}$$

where the factor 1.4826 converts the median of the absolute values to a Gaussian σ and `network.raim_sigma0_floor` (default 1.0) is a lower bound. The pool holds, for each carried-ambiguity column of a station, the rows whose coefficient on the column exceeds 0.5 and whose kind the row classifier admits: the observation phase rows, the coupling and block carried-prior rows and the SD-EWL constraint rows (`raimPoolAdmitsKind`, `network/assembly/RowClassify.hpp`). `network.raim_pool_data_rows_only` (default off) restricts it to the observation phase rows, and `network.raim_redundancy_normalize` (default off) divides each row by $\sqrt{r_i}$ and drops the rows below the redundancy floor. Unlike the Kalman variance factor of Equation (6), $\hat{\sigma}_{0,\mathrm{net}}$ is *not* used for reporting only. It controls three things:

- a within-epoch RAIM adaptation loop, whenever it exceeds `network.raim_sigma0_trigger` (default 3.0);
- a whole-epoch and a per-constellation product withhold, above `network.raim_withhold_sigma0`;
- the scale of the emitted URA.

Three separate seams implement them. The within-epoch loop and the ceiling are stages 13 and 14 of the epoch's solve (`networkWithinEpochAdapt` and `networkSigma0Ceiling`, `network/qc/RaimWithinEpoch.cpp`; the per-constellation decision is `NetworkEstimator::decidePerConstWithhold`). `networkPostFitRaim` (`network/qc/Raim.cpp`) owns the post-fit pass and the exponentially weighted fold of $\hat{\sigma}_{0,\mathrm{net}}$ into the emitted URA scale. `networkSolPrep` (`network/emit/SolPrep.cpp`) owns the emission-side prune that removes a withheld constellation's satellites. Sections 1.9 and 1.10 develop the adaptation loop and the pooled statistics. The emitted product is not treated here.

1.3 Redundancy numbers, the Baarda w-test, and the MDB

The detection theory rests on the post-fit residual covariance $\boldsymbol{Q}$ (the covariance of $\hat{\boldsymbol{v}}$),

$$\boldsymbol{Q} \;=\; \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}^{+}\,\boldsymbol{H}^{\mathrm{T}}, \tag{10}$$

where $\boldsymbol{P}^{+}$ is the *updated* (post-fit) state covariance. `KalmanFilter::update` builds the diagonal $\boldsymbol{Q}_{ii}$ per row as $R_{ii} - (\boldsymbol{H}\boldsymbol{P}^{+}\boldsymbol{H}^{\mathrm{T}})_{ii}$. The full matrix is formed only under the off-diagonal kernel of Section 1.5.

Redundancy number. The (local) redundancy of observation i is

$$\boxed{r_i \;=\; \frac{\boldsymbol{Q}_{ii}}{R_{ii}} \;=\; \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i \in [0,1],} \tag{11}$$

clamped to $[0,1]$ for numerical safety in both branches of `KalmanFilter::residualQcKernel` (called by `computeResidualQc_`). When $r_i \to 1$ the geometry checks the observation fully, and a

blunder appears almost entirely in the residual. When $r_i \to 0$ the observation is *uncheckable*: the estimate absorbs a blunder and the residual shows nothing. The sum dof $= \sum_i r_i = \operatorname{tr} \boldsymbol{\Theta}$ is the effective number of degrees of freedom used in Equation (6).

The network estimator computes Equation (11) in the batch (arrowhead) algebra of the Schur-conditioned shared covariance, $r_i = 1 - w_i \, \boldsymbol{a}_i^{\mathrm{T}} \operatorname{Cov}(\boldsymbol{x}) \, \boldsymbol{a}_i$ with $\boldsymbol{a}_i = [\, \boldsymbol{a}_{s,i} \mid \boldsymbol{a}_{r,i} \,]$ and the same $[0, 1]$ clamp. This is algebraically the batch form of $r_i = \boldsymbol{Q}_{ii}/R_{ii}$ (`NetworkNormalEquations::rowRedundancy`, gated by `network.raim_redundancy_normalize`, default off). Rows at or below the redundancy floor 10^{-6} (`kBaardaRedundancyFloor`) are uncheckable and are dropped from the $\hat{\sigma}_{0,\mathrm{net}}$ pool of Equation (9). This corresponds to the `kUncheckableRedundancy` cutoff of the KF snooping gate below. The network constant is much smaller, because it protects only the $\sqrt{r_i}$ standardisation and not a reset decision.

One case of $r_i = 0$ is exact, and it determines the design of the network's phase-side QC. Suppose each (station, satellite, signal) arc has its own free ambiguity, which is the case whenever `network.phase_leveling` is off. Then every phase row is fitted *exactly* by its own parameter. The post-fit phase residual is identically zero, r_i is exactly zero, and the residuals cannot reveal a phase fault of any size. An undifferenced network with per-arc float ambiguities has *no* phase-side redundancy.

The phase rows gain redundancy, and so the power to detect faults, only when a carried prior constrains the ambiguities. That is why the network's phase RAIM (Section 1.9) requires phase-leveling: without it the test has no power at all. The measure-only instrument `networkPostfitSigma0` (`network/diag/FrameDiag.cpp`) reports the effect. Its pooled phase variance factor is identically zero in that configuration, and the zero is the correct result. Its pools exclude two classes of row for related reasons. Rows that touch a shared spherical-harmonic column (the optional ionosphere field, Section 1.10) are pseudo-observations, not data. The carry-coupling rows describe the carry, not the epoch's measurements.

w-test. Baarda's data-snooping statistic standardises each residual by its own post-fit standard deviation. The diagonal (uncorrelated) form is

$$w_i \;=\; \frac{\hat{v}_i}{\sqrt{\boldsymbol{Q}_{ii}}} \tag{12}$$

(the diagonal branch of `KalmanFilter::residualQcKernel`). The divisor is $\sqrt{\boldsymbol{Q}_{ii}}$, the *true* residual standard deviation, not $\sqrt{R_{ii}}$. With $\sigma_0^2 = 1$ known (every $\boldsymbol{R}$ here is absolute), this is the standardised residual of [2], chapter 9; it becomes the studentised residual when an estimated $\hat{\sigma}_0$ replaces σ_0. Under the null hypothesis $w_i \sim \mathcal{N}(0, 1)$. The s used by the fixed gate floors the denominator at R and is therefore conservative. When the observations are correlated, the weight matrix $\boldsymbol{R}^{-1}$ takes the place of $1/R_{ii}$ (the `use_full` branch of the same routine):

$$\boxed{w_i \;=\; \frac{\left(\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}\right)_i}{\sqrt{\left(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1}\right)_{ii}}}.} \tag{13}$$

This is Baarda's statistic for a single blunder in observation i [1, 6], the most powerful test of that alternative, and the form of the LASER guide's data snooping [3]. In a Kalman update $\boldsymbol{R}^{-1}\hat{\boldsymbol{v}} = \boldsymbol{S}^{-1}\boldsymbol{z}$ and $\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1} = \boldsymbol{S}^{-1}$, with $\boldsymbol{z}$ the innovation and $\boldsymbol{S}$ its covariance, so Equation (13) is also the innovation w-test $(\boldsymbol{S}^{-1}\boldsymbol{z})_i/\sqrt{(\boldsymbol{S}^{-1})_{ii}}$ [5, 7]. It reduces to Equation (12) when $\boldsymbol{R}$ is diagonal, although $\boldsymbol{Q}$ is not diagonal in a Kalman update. Until 2026-10-07 the branch used $\boldsymbol{Q}^{-1}$ in place of $\boldsymbol{R}^{-1}$. That statistic is also standard normal under the null hypothesis, but it tests the direction $\boldsymbol{S}\boldsymbol{R}^{-1}\boldsymbol{e}_i$ rather than observation i: it lost power against a single blunder, and it did not

reduce to Equation (12) even for a diagonal $\boldsymbol{R}$. `tests/test_residual_qc_wtest.cpp` checks the routine against a batch Gauss–Markov model with the prior as stochastic constraints.

Minimal detectable bias. The smallest blunder on observation i that the w-test will detect, at the configured significance and a conventional 80% test power, is

$$\boxed{\mathrm{MDB}_i \;=\; \frac{\sigma_i\,\delta_0}{\sqrt{r_i}}, \qquad \delta_0 = \texttt{dataSnoopingCritical()} + 0.8416,} \tag{14}$$

with $\sigma_i = \sqrt{R_{ii}}$ the a-priori observation σ (the diagonal branch of `KalmanFilter::residualQcKernel`). With correlated observations the same definition gives $\mathrm{MDB}_i = \delta_0/\sqrt{(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1})_{ii}}$, so that a blunder of size MDB_i in observation i shifts the expectation of Equation (13) by exactly δ_0; it equals Equation (14) when $\boldsymbol{R}$ is diagonal. The non-centrality δ_0 is the critical value plus $\Phi^{-1}(0.80) = 0.8416$, for a test power of 80% ($\beta = 0.20$). For the default critical value $k = 3.29$ this gives $\delta_0 \approx 4.13$. The factor $1/\sqrt{r_i}$ expresses the geometry. A poorly checked observation ($r_i \to 0$) has an arbitrarily large MDB, and the code returns $+\infty$ when r_i is at or below the floor 10^{-6} (`kBaardaRedundancyFloor`). A blunder on such a row cannot be detected from the residual. The network estimator computes the same quantity from its own redundancy numbers (`network/qc/Mdb`, `network.mdb`), with $\sigma_i = 1/\sqrt{w_i}$. Both paths take δ_0, the redundancy floor and the $+\infty$ convention from one header, `filter/BaardaReliability.hpp`, so a PPP MDB and a network MDB are commensurable. The network MDB is measure-only: no gate, deweight or withhold reads it. Its loader default is off, and both reference configurations enable it. The network's acting phase-side QC remains the robust $\hat{\sigma}_{0,\mathrm{net}}$ RAIM loop of Equation (9).

1.4 Data-snooping gate override

When `enableDataSnooping()` is set, the QC statistics drive an *alternative* outlier gate, which can select a row that the fixed $|s| > 6$ path missed. Both gates use the same downstream dispatch in `KalmanFilter::validatePostFitResiduals`. Only the *statistic, index, and critical value* differ. By default the gate is seeded from the fixed-σ triple $(\max_i |s_i|, \arg\max, 6)$, so the dispatch is byte-identical when snooping is off.

When snooping is enabled the gate is seeded from the w-test over *checkable* rows only. Rows with redundancy below the `kUncheckableRedundancy` cutoff are skipped. Their residuals are unreliable, and resetting the corresponding ambiguity would discard good information. The same cutoff filters the summary log, so the log and the gate agree. Over the remaining rows the gate fires on

$$\max_{\{i\,:\,r_i \geq \texttt{kUncheckableRedundancy}\}} |w_i| \;>\; \texttt{dataSnoopingCritical()}, \tag{15}$$

and the checkable row with the largest $|w_i|$ becomes the handled index. w_i uses the true residual σ (Equation (12)), not the R-floored s. The gate can therefore catch a blunder on a high-leverage row whose floored s stayed below 6, and in that case it overrides the s choice. The downstream Melbourne–Wübbena tiebreaker still works in the σ domain. When the w-test selects a row that the σ cut missed, no σ candidate exists, and the w-selected index is handled directly. The fixed standardised-residual path is described in the gating section.

1.5 Correlation-aware full-$\boldsymbol{R}$ kernel

The diagonal forms above assume that $\boldsymbol{R}$ and $\boldsymbol{Q}$ are diagonal, which is true for undifferenced PPP. For BSSD and baseline-DD epochs the differencing operator $\boldsymbol{D}$ of the differencing transformations

turns the diagonal undifferenced covariance into a *dense* one, $\boldsymbol{R}_{\mathrm{SD/DD}} = \boldsymbol{D}\,\boldsymbol{R}_{\mathrm{ZD}}\,\boldsymbol{D}^{\mathrm{T}}$. It contains the cross-covariance between observations that share a pivot or reference. The off-diagonal kernel (`estimation.enable_full_r_kernel`, default off) also requires that a dense $\boldsymbol{R}$ was supplied for the epoch, which `KalmanFilter::update` checks with `dense_r_supplied`. Under it the full residual covariance Equation (10) is built densely and symmetrised, and the QC quantities follow without approximation:

$$\boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i = \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \mathrm{dof} = \operatorname{tr}\boldsymbol{\Theta}, \tag{16}$$

$$\Omega = \hat{\boldsymbol{v}}^{\mathrm{T}}\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}, \qquad w_i = \frac{\big(\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}\big)_i}{\sqrt{(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1})_{ii}}}, \qquad \mathrm{MDB}_i = \frac{\delta_0}{\sqrt{(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1})_{ii}}}. \tag{17}$$

$\boldsymbol{R}^{-1}$ comes from an $\mathrm{LDL}^{\mathrm{T}}$ solve against the identity. An $\boldsymbol{R}$ that is not positive definite has no weight matrix, and the routine then falls back to the per-row forms. When $\boldsymbol{R}$ is diagonal, $\boldsymbol{\Theta}$, w and MDB equal the per-row quantities of Section 1.3, whatever the off-diagonals of $\boldsymbol{Q}$. The diagonal of the full $\boldsymbol{Q}$ equals the per-row $\boldsymbol{Q}_{ii}$, so the fixed $|s| > 6$ gate is *unchanged behind the flag*; under data snooping (Section 1.4) the gate reads the correlation-aware w_i. The flag only makes the statistics correct on differenced epochs, where the residuals are correlated. Its scope is narrow. The gain and the Joseph-form update already use a dense $\boldsymbol{R}$ whenever one is supplied, with or without the flag. The flag decides only whether the *post-fit residual* covariance is formed densely. The differencing section describes the corresponding correlation on the model side.

1.6 Block Baarda test on a line of sight

One physical fault on one line of sight (multipath, an unflagged slip) enters the code and phase rows of every band at once, and the adjustment spreads it across all of them. The per-row w-test sits below that unit and the station-level median polish above it, so neither names the line of sight. The block test takes the b rows of one (station, satellite) block B and forms Baarda's multi-dimensional statistic for the alternative "an arbitrary bias vector on the block" [3, 6]. With $\tilde{\boldsymbol{e}}_B$ the post-fit residuals of the block and $\boldsymbol{Q}_B$ their $b \times b$ cofactor, off-diagonals kept (`NetworkNormalEquations::blockResidualCofactor`),

$$T_B \;=\; \frac{\tilde{\boldsymbol{e}}_B^{\mathrm{T}}\,\boldsymbol{Q}_B^{-1}\,\tilde{\boldsymbol{e}}_B}{b}, \qquad t_B \;=\; \frac{T_B}{\hat{\sigma}_{0,f}^2}. \tag{18}$$

With a diagonal weight matrix the weight factors of the general form cancel, so for $b = 1$ it is the square of the scalar w-test, and under the null $b\,T_B \sim \sigma_0^2\chi_b^2$, so $E\{t_B\} = 1$ for every block size. $\hat{\sigma}_{0,f}^2$ is a calibration per constellation family: the tail-robust geometric-mean estimator of Section 1.7, fed the χ_1^2-equivalent $u_B = \sqrt{T_B\,b\,e^{\psi(1/2)}/e^{\psi(b/2)}}$ of each block, so that a mix of block sizes does not bias it. The calibration is frozen through the previous epoch before the epoch's blocks feed it, every checkable block feeds it before a threshold is consulted, and it is not clamped. The block survey (`network/qc/BlockBaarda.hpp`, `network.block_baarda`) is measure-only: no weight, state or decision reads it.

Estimable form. On the float vehicle a line of sight's slant ionosphere is a free column that no row outside the block touches, so it absorbs one direction of the block exactly. The cofactor $\boldsymbol{Q}_B$ is then singular however well the data are conditioned, and the whole-block test refuses the block. The estimable form (`network.block_baarda_estimable`) tests what the data can test. Whiten the cofactor with the row weights, $\tilde{\boldsymbol{Q}} = \boldsymbol{W}^{1/2}\boldsymbol{Q}_B\boldsymbol{W}^{1/2} = \boldsymbol{I} - \boldsymbol{H}_{BB}$, whose eigenpairs $(\lambda_k, \boldsymbol{v}_k)$ have

eigenvalues that are redundancy numbers in $[0,1]$; keep the directions with $\lambda_k \geq \varepsilon$ (`network.block_baarda_estimable_eps`, default 0.05, the row survey's floor) and read the quadratic form on them:

$$\tilde{\boldsymbol{e}}_B^{\mathrm{T}} \boldsymbol{Q}_B^{+} \tilde{\boldsymbol{e}}_B \;=\; \sum_{\lambda_k \geq \varepsilon} \frac{\left(\boldsymbol{v}_k^{\mathrm{T}} \boldsymbol{W}^{1/2} \tilde{\boldsymbol{e}}_B\right)^2}{\lambda_k}, \qquad T_B \;=\; \frac{\tilde{\boldsymbol{e}}_B^{\mathrm{T}} \boldsymbol{Q}_B^{+} \tilde{\boldsymbol{e}}_B}{m}, \quad m = \#\{\lambda_k \geq \varepsilon\}. \tag{19}$$

Under the null this is $\sigma_0^2 \chi^2_m$ with the kept count m as its degrees of freedom, which a plain pseudo-inverse lacks, and the scale correction above is taken at m. When nothing is dropped the statistic is the whole-block one. A fault that lies wholly in a dropped direction (an additive slant-ionosphere change is the example) is absorbed by the column that makes the direction null, and no test of the block can see it; this one does not claim to. Both settings are armed in the two reference configurations. The positive control, a 1.5 m code multipath injected on one line of sight, is flagged by the estimable form at 24 times the control's rate and by no other armed check (`docs/record/refound/BBE_block_baarda_estimable_2026-10-18.md`, section 9).

1.7 Variance components of the carried priors

The network links epochs with carried priors whose variances are set by hand, a floor and a rate per carry family: the phase-leveling ambiguity carry (`pl`), the shared satellite phase-bias carry (`pb`) and the shared satellite code-bias carry (`cb`). A monitor (`NetworkCarryVce`, `network/carry/CarryVce.hpp`; `network.carry_vce`, loader default off, armed on the three families in both reference configurations) checks them against the carry residuals. Each epoch a carried quantity is folded as a pseudo-observation of the state with prior mean $\bar{b}_i$ and prior variance $p_i = \max(\mathrm{var}_{i,\mathrm{prev}} + q\,\Delta t,\ \mathrm{floor})$. After the solve the state has value $\hat{x}_i$ and posterior variance v_i. The carry-row innovation

$$\hat{e}_i \;=\; \bar{b}_i - \hat{x}_i, \qquad r_i \;=\; 1 - w_i v_i \in [0,1], \quad w_i = 1/p_i, \tag{20}$$

does not depend on how the fold is represented (a diagonal prior or the coupling rows), and r_i is the partial redundancy of the row. The monitor models the carry-row variance as a floor component (cofactor 1, m^2) plus a rate component (cofactor Δt_i, the arc gap, in m^2/s) per (family, constellation), and estimates both by residual-form least-squares variance-component estimation at the weight the solve used [8]:

$$N_{kl} \mathrel{+}= w_i^2\, c_i^{(k)} c_i^{(l)}\, r_i, \qquad l_k \mathrel{+}= w_i^2\, c_i^{(k)}\, \hat{e}_i^2, \qquad \hat{\boldsymbol{\sigma}} = \boldsymbol{N}^{-1}\boldsymbol{l}. \tag{21}$$

The redundancy is taken block-diagonal in the carry rows. The two cofactors are collinear when every arc gap equals the decimation interval, so a weak a-priori regularises $\boldsymbol{N}$ and its condition number is reported. Its primary output is a tail-robust geometric mean of $\hat{e}_i^2$ over a forgetting window (`network.carry_vce_window_epochs`, default 60), corrected by the geometric mean $e^{-\gamma}/2 \approx 0.2807$ of a χ^2_1 variate, and a second channel formed from $\hat{e}_i^2/r_i$. Contaminated carries (a slip or refloat, a datum re-reference, a handover, a RAIM exclusion) are withheld from the accumulators. As a monitor it never feeds back into the solve.

The feedback is `network.carry_vce_apply` (`network/carry/CarryVceApply.hpp`), built and default off, not armed in either reference configuration. It multiplies the fold variance p_i of each carry family and constellation by a factor $\alpha^2 \geq 1$, updated by the variance-factor iteration of the LASER guide (slides 5-93 to 5-95) [3], $\alpha^2_{j+1} = \hat{\sigma}_j^2\, \alpha_j^2$. The group estimate $\hat{\sigma}^2$ divides the weighted residual square sum by the group's redundancy [2]. A carry row folded at $\alpha^2 p_i$ with posterior variance v_i has the partial redundancy $r_i = 1 - v_i/(\alpha^2 p_i)$ and $E\{\hat{e}_i^2\} = r_i\, \alpha^2 p_i\, \hat{\sigma}^2$. The streaming target is therefore formed row by row, as a geometric mean consistent with the monitor's:

$$\alpha^2 \;\leftarrow\; \frac{\exp\!\left(\overline{\log(\hat{e}_i^2/r_i)}\right)/c_{\chi^2_1}}{\exp\!\left(\overline{\log p_i}\right)}, \qquad c_{\chi^2_1} = e^{-\gamma}/2 \approx 0.2807, \tag{22}$$

where the overbar is the monitor's forgetting-weighted mean, p_i is the hand fold variance before the factor, and $c_{\chi^2_1}$ is the geometric mean of a χ^2_1 variate. A row with r_i at or below 10^{-3}, or with $r_i < 0$, carries no information on the factor and is left out and counted. The case $r_i < 0$ arises on the coupled ambiguity carry, whose harvested variance is a conditional variance that can exceed the prior; there r_i is an approximation. The step toward the target is clamp(target/α^2, $[1/s, s]$) with the slew limit s (`network.carry_vce_apply_slew`, default 1.05 per epoch), and α^2 is clamped to [1, `network.carry_vce_apply_max`]. The factor is inflate-only, slew-limited, frozen on a contaminated epoch or below a warm-up row count, and valid only from a fresh convergence. Until 2026-10-07 the target omitted the division by r_i. For a scalar carry with true prior variance $a\,p$ and data variance d, $E\{\hat{e}^2\} = r^2(ap + d)$, so that target settled below a, and once d exceeded about $ap/4$ it stayed at $\alpha^2 = 1$: the under-inflation was largest where the carry is stiff against its data.

1.8 Measure-only instruments of the network

Four further instruments read the network's data and priors without acting on them. Each is off in the loader and armed in both reference configurations; none changes a weight, a state or a decision.

- *Prefit survey* (`network/qc/PrefitCensus.hpp`). A raw prefit innovation of 10^5 m is not a fault: a millisecond receiver clock step puts it on every row of a station. The statistic is centred per station and row type, $s_i = |y_i - m_t|\sqrt{w_i}$ with m_t the median of the innovations y_i of type t, a between-satellite single difference free of the receiver clock. It flags magnitude on established cells; a cell born in the epoch is passed to the seed-birth tripwire.
- *Redundancy survey* (`network/qc/RedundancyCensus.hpp`). The row redundancy $r_i = 1 - w_i \boldsymbol{a}_i^{\mathrm{T}} \mathrm{Cov}(\boldsymbol{\theta}) \boldsymbol{a}_i$ says how much of a row is cross-checked by the rest of the network. A row with $r_i \to 0$ only feeds a state and its misfit is invisible to every residual test, whatever its size. The alarm is the cross-flag of an uncheckable row ($r_i < \varepsilon$) with a large centred innovation ($s_i > S$). The sweep runs every `network.redundancy_census_stride` epochs (10 in the reference configurations) and hosts the network MDB, the block test and the congruence screen.
- *Prior-residual survey* (`network/qc/PriorResidual.hpp`). The residual of a stochastic constraint in the LASER guide's form (slide 5-74), for every carried stiff prior. For a prior folded into the solve with variance p and posterior variance v, $D\{\hat{e}\} = r\,p = p - v$; for a prior the epoch did not fold, $D\{\hat{e}\} = p + v$. The standardised value is $s = \hat{e}/\sqrt{D\{\hat{e}\}}$ and the empirical one $t = s/\hat{\sigma}_{0,f}$. It is the carry-row counterpart of the per-row w-test of Section 1.3, on the quantities of Section 1.7.
- *Congruence screen* (`network/qc/Congruence.hpp`). Station coordinates are held fixed, so an error in a held coordinate projects coherently into every row of the station and the other tiers are blind to it. The screen estimates the coordinate offset that best explains the station's own post-fit code and phase residuals and tests it against zero,

$$M = \boldsymbol{H}_B^{\mathrm{T}} \boldsymbol{P}_B \boldsymbol{Q}_B \boldsymbol{P}_B \boldsymbol{H}_B, \qquad \hat{\boldsymbol{\delta}} = M^{+} \boldsymbol{H}_B^{\mathrm{T}} \boldsymbol{P}_B \tilde{\boldsymbol{e}}_B, \qquad T = \frac{\hat{\boldsymbol{\delta}}^{\mathrm{T}} M \hat{\boldsymbol{\delta}}}{3\,\hat{\sigma}_0^2}, \tag{23}$$

  with $\boldsymbol{H}_B$ the $b \times 3$ position Jacobian in east, north and up and a coordinate MDB $\delta_0 \hat{\sigma}_0 \sqrt{(M^{+})_{kk}}$ per direction. It detects a held-coordinate error and cannot size one.

1.9 Network RAIM: within-epoch adaptation, withhold and post-fit tiers

The network checks its own epoch at three points, in this order: the within-epoch adaptation inside the solve, the $\hat{\sigma}_{0,\mathrm{net}}$ ceiling that can withhold the epoch, and the post-fit tiers after the carry harvest.

The doctrine is to adapt and emit within the detected epoch; a withhold is the backstop, and it counts as an adaptation failure unless the epoch is provably unrecoverable.

Within-epoch adaptation. When $\hat{\sigma}_{0,\mathrm{net}}$ exceeds its trigger, the network chooses the *granularity* of its response from a robust two-way decomposition of the epoch's standardised residuals. Taking the largest standardised residual m_{ri} of each (station r, satellite i) carried-ambiguity cell as a sparse two-way table and applying a Tukey median polish,

$$m_{ri} \approx \mu + \alpha_r + \beta_i + \gamma_{ri}, \tag{24}$$

splits the contamination into a grand effect μ, a per-station row effect α_r, a per-satellite column effect β_i, and an isolated-arc interaction γ_{ri} (`medianPolish2Way`, `network/qc/RobustStats.hpp`). The polish sweeps medians, not means, so a minority of very large cells (the outliers) does not distort the additive effects. A bad satellite, which raises a whole column, can then be told apart from a bad station, which raises a whole row. Isolated arc slips remain in the interaction term.

Each effect is a candidate culprit. To be eligible it must satisfy effect$/\hat{\sigma}_{0,\mathrm{net}} > k_{\mathrm{RAIM}}$ for the configured gate (`network.raim_k_sigma`, default 6.0). The eligible candidates are ranked with the narrowest response first: on equal score an arc ranks above a satellite, and a satellite above a station (`weChooseCulprit`). The station branch is also reduced by a bias factor (`network.raim_station_drop_bias`, default 1.5, active under `network.raim_surgical_station`, default on). Excluding a station discards good arcs with the bad, so it is chosen only when it dominates by that margin. A consensus rule avoids removing arcs one at a time. When at least `network.raim_min_stations` (default 3) of one satellite's arcs exceed the gate in one iteration, the arc candidate is replaced by the whole satellite. A corrupted *shared* clock or phase bias is a network-wide fault, not a set of independent local ones.

The response distinguishes a bad *measurement* from a stale *parameter*. This distinction is the central design decision. A station culprit is excluded, because its data are bad and exclusion is the right response to bad data. A satellite or arc culprit is almost always a cycle slip. A slipped ambiguity does not need less weight; it needs a new *value*. The epoch is therefore solved again on a rebuilt normal-equation system with the culprit's ambiguities *refloated*. Their measurements keep full weight, so the local block stays positive definite and no singularity is introduced. Their carried prior is withheld, so the new solve can find the new value and absorb the slip locally. The loop repeats until $\hat{\sigma}_{0,\mathrm{net}}$ falls below the trigger or five passes have run. A pass whose re-solve fails is undone to the last landed state; a station the undo cannot re-add is left out of the epoch, and an undo that cannot complete skips the epoch. Only the rare contaminated epochs pay for a second solve.

`network.raim` and `network.raim_within_epoch` both default to false and are armed in the two reference configurations. *Both* tiers also require the phase-leveling carry to be warm: the within-epoch loop inside the solve, and the post-fit pass at its call site. With `network.phase_leveling` off there are no carried ambiguities to refloat. By the redundancy argument of Section 1.3 there is also no phase-side redundancy to test. The network's RAIM then does nothing, as a consequence of the model and not of a policy choice.

The ceiling and the withhold. The capped adaptation may not isolate the fault. With `network.raim_withhold_sigma0` positive (default 0, off; 6.0 in both reference configurations), an epoch whose final $\hat{\sigma}_{0,\mathrm{net}}$ still exceeds the ceiling is withheld: it is not emitted, its carry is not stashed and the epoch is skipped, so the contaminated solution never enters the product store or the carry. The ceiling is a backstop. On the 6 h reference runs it never fired (720 of 720 epochs emitted on each

of the real-data and simulated cases; `docs/CONFIG_promotion_ledger.json`, row `network.raim_withhold_sigma0`). Two extensions of the withhold are built and unvalidated: a larger adaptation budget inside a run of consecutive withheld epochs (`network.raim_withhold_adapt_escalate`) and the erase of detector-freed carries at a withhold (`network.raim_withhold_free_erase`).

Post-fit tiers. After the joint solve and the carry harvest, each station is reconstructed once (`networkPostFitRaim`) and its carried-ambiguity residuals are standardised by $\hat{\sigma}_{0,\mathrm{net}}$ of the same pool. Two screens look for slips the detectors missed. The consensus screen refloats a satellite whose standardised residual exceeds k_{RAIM} at `network.raim_min_stations` or more stations network-wide. The attribution screen (`network.raim_attribution`, armed) applies the median polish of Equation (24) and drives the refloat in place of the consensus: a bad satellite (column effect over the bar) is refloated network-wide only when $\hat{\sigma}_{0,\mathrm{net}}$ exceeds the trigger, because on a clean epoch a high column effect is an artifact of a satellite seen by a regional cluster; an isolated arc (interaction over the bar) is refloated at any $\hat{\sigma}_{0,\mathrm{net}}$. A bad station is never refloated wholesale, since a cluster of stations raised by one shared satellite event inflates their row effects and the cascade diverged the solution; it is handled by the within-epoch station exclusion or surfaces as isolated arcs. On the float vehicle the refloat of an arc is a bounded drop of each band's leveling carry, value kept and variance returned to its birth strength.

Refloat maturity gate. An isolated-arc flag meets two gates in this order before it refloats the arc. The maturity gate (`network.raim_refloat_maturity_gate`, unvalidated, off in the reference configurations) skips the refloat until the run is mature: until it has emitted $\max(N_{\mathrm{mat}}, N_{\mathrm{warm}})$ epochs and the $\hat{\sigma}_{0,\mathrm{net}}$ average that scales the emitted URA has received N_{mat} updates, with N_{mat} = `network.raim_refloat_maturity_epochs` (default 0, which leaves the gate inert) and N_{warm} the carry warm-up (`network.phase_leveling_warmup_epochs`). Without `network.emit_ura` the average is never fed and the epoch count alone decides. The persistence gate (`network.raim_arc_persist_epochs` $= K$, default 1) refloats an arc only on its K-th consecutive flagged epoch. Whole-satellite flags are never gated. The maturity gate changes which information reaches the carried ambiguities, so it is an event at both edges. Each held flag is an arc-epoch on which the refloat did not run, and the *lift*, the first scan with the gate open, is where the test applies to every arc again. Both are counted and written as events of tier `RAIM_MATURITY` (one row per held flag with the arc's identity and one at the lift), with a run-end section and an arming line; the `acted` column of the attribution trace reads a refloat taken, not a flag raised. Graded on 6 h cases, the gate at 120 epochs halves the isolated-arc refloats (107 to 55) and moves no user by more than 0.1 mm; it stays off (`docs/record/refound/RCG_refloat_cutting_keys_2026-10-29.md`).

Signal-integrity classifier. The second setting that cuts refloats is `network.signal_integrity.census` (unvalidated, off). It classifies each line of sight each epoch (stable, new, re-entry, band regained, degraded), folds the phase rows of a line of sight that has not been vetted at a tenfold standard deviation until it has had `revet_ep` consecutive full-band epochs, and refloats the ambiguities of a line of sight whose reference band returns within `flicker_max_ep` epochs. It detects nothing: across a loss of lock the slip is unbounded and unobservable, so it withholds trust from the lines of sight no detector can monitor. It cut the isolated-arc refloats from 107 to 78 but the slip-class responses by only 3.3 % (same document), so it stays off.

1.10 Pooled variance factors and the per-constellation carve-out

Equation (9) pools only the carried-ambiguity rows. Two extensions widen or partition that set of rows without changing the estimator. Both are off by default.

Code and iono-constraint pools. Originally no tier screened the code rows or the iono-constraint rows that tie each line of sight to a spherical-harmonic field. Under `network.raim_we_code_iono` the same scaled-median estimator is applied to each of them over its own disjoint set of rows,

$$\hat{\sigma}_{0,\text{code}} = 1.4826\ \text{median}_{i\in\mathcal{C}}\,\tilde{v}_i, \qquad \hat{\sigma}_{0,\text{iono}} = 1.4826\ \text{median}_{i\in\mathcal{I}}\,\tilde{v}_i, \tag{25}$$

each with its own trigger (`network.raim_we_code_trigger`, default 10; `network.raim_we_iono_trigger`, default 12). Every row belongs to exactly one of the three sets. Neither new pool has the floor of Equation (9), and the phase pool still drives the whole-epoch ceiling. The adaptation gains two responses suited to what these rows can diagnose: a station exclusion for the epoch, and a network-wide *deweighting* of one satellite's code rows. The second is a fallback and not a drop, in line with the project's preference for degrading a measurement over discarding it. The setting is off in the reference configurations: it never tripped on their posture, and its iono pool is empty there, because the tie rows exist only when the optional spherical-harmonic field is estimated. The reference configurations estimate no such field, and the station-relative deweighting of its tie rows was deleted from the code.

Per-constellation carve-out. When the global ceiling trips, the default response withholds the whole epoch's product. Under `network.per_const_withhold` (unvalidated, off) the same pool is instead partitioned by the ambiguity's satellite constellation and each part judged on its own,

$$\hat{\sigma}_{0,c} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \text{median}\{\tilde{v}_i : i \in \mathcal{P}_c\}\big), \tag{26}$$

over the same rows $\mathcal{P}_c$ that the global statistic pools. The statistic and the ceiling are the same, and only the set of rows is restricted. A constellation is published if and only if $\hat{\sigma}_{0,c}$ is below that ceiling *and* $|\mathcal{P}_c|$ meets a minimum row count (`network.per_const_withhold_min_rows`, default 20). Too few rows cannot show that a constellation is clean inside a contaminated epoch (`NetworkEstimator::decidePerConstWithhold`).

This is an approximation, in two respects. The residuals in Equation (26) are evaluated at the *joint* solution. A contamination in one constellation perturbs the shared per-station nuisance parameters (receiver clock, tropospheric delay, slant ionosphere) and leaks into the residuals of another. Each row's redundancy is also its share of the joint system's redundancy, not of a separate per-constellation solve. $\hat{\sigma}_{0,c}$ therefore under-separates coupling between constellations. The outcome "every constellation individually clean while the global statistic tripped" is a consequence of that approximation. It is published with a warning and counted. The rank and observability gate keeps whole-epoch semantics. A rank deficiency invalidates the declared datum, so the product would be a meaningless least-norm split and no healthy subset exists.

1.11 Emission-time gates

Everything above tests *residuals* against $\boldsymbol{R}$. The network also prunes the emitted product at emission time (`networkSolPrep`, `network/emit/SolPrep.cpp`). The per-constellation carve-out of Section 1.10 is applied there, and so are the satellite-level exclusions of the gross line-of-sight screen (the gross line-of-sight screen). A different kind of gate tests the *solved state*.

Clock-runaway quarantine. Under `network.clock_runaway_check` (loader default off; armed in the two reference configurations) the epoch's satellite clock solutions are grouped by constellation, and a constellation is dropped from the emission when the median of $|\text{clock} - \text{median}_{\text{all}}|$ exceeds a magnitude ceiling (`network.clock_runaway_mag_m`, default 1000 m), or the median |drift| exceeds a rate ceiling (`network.clock_runaway_rate_m_s`, default 10 m/s, inert without `network.clock_model`). Like the carve-out of Section 1.10, this reduces the scope of the product and does not withhold all of it. The code documents one hazard and does not defend against it. The magnitude test is referenced to the global median over *all* clock satellites. If the runaway constellation supplies half or more of them, the comparison inverts: the healthy minority exceeds the ceiling and is quarantined, and the runaway passes. The degenerate single-constellation case is warned at configuration load. The check has not been recorded firing on the reference runs.

Clock quality control. `networkClockQc` (`network/qc/ClockQc.cpp`) is a different tier. It contains the two-state oscillator model with its normalised-innovation fault test, the decoupled emit-clock-rate estimator, and the per-satellite sanity monitor for carried clocks. The health gate on the spherical-harmonic field that once sat beside the quarantine was deleted from the code. The field itself remains as a default-off option that the reference configurations do not use.

References

[1] W. Baarda. *A Testing Procedure for Use in Geodetic Networks*. Vol. 2. Publications on Geodesy, New Series 5. Delft, The Netherlands: Netherlands Geodetic Commission, 1968.

[2] K. Snow. *Adjustment Computations*. Lecture notes based on courses GS 650, GS 651 and GS 762 taught by B. Schaffrin at The Ohio State University. Jan. 2026.

[3] K. Snow, F. Yang, and A. Whitacre. *A Guide to LASER: LSA Libraries and Programs, Version 2.7*. Polaris Geospatial Services. Columbus, Ohio, Aug. 2024.

[4] J. Sanz Subirana, J. M. Juan Zornoza, and M. Hernández-Pajares. *GNSS Data Processing, Volume I: Fundamentals and Algorithms*. ESA TM-23/I. Noordwijk, The Netherlands: ESA Communications, 2013.

[5] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.

[6] P. J. G. Teunissen. *Adjustment Theory: An Introduction*. Series on Mathematical Geodesy and Positioning. Delft, The Netherlands: Delft University Press, 2000.

[7] P. J. G. Teunissen and O. Montenbruck, eds. *Springer Handbook of Global Navigation Satellite Systems*. Cham, Switzerland: Springer, 2017. ISBN: 978-3-319-42926-7. DOI: 10.1007/978-3-319-42928-1.

[8] Peter J. G. Teunissen and A. R. Amiri-Simkooei. "Least-squares variance component estimation". In: *Journal of Geodesy* 82.2 (2008), pp. 65–82. DOI: 10.1007/s00190-007-0157-x.